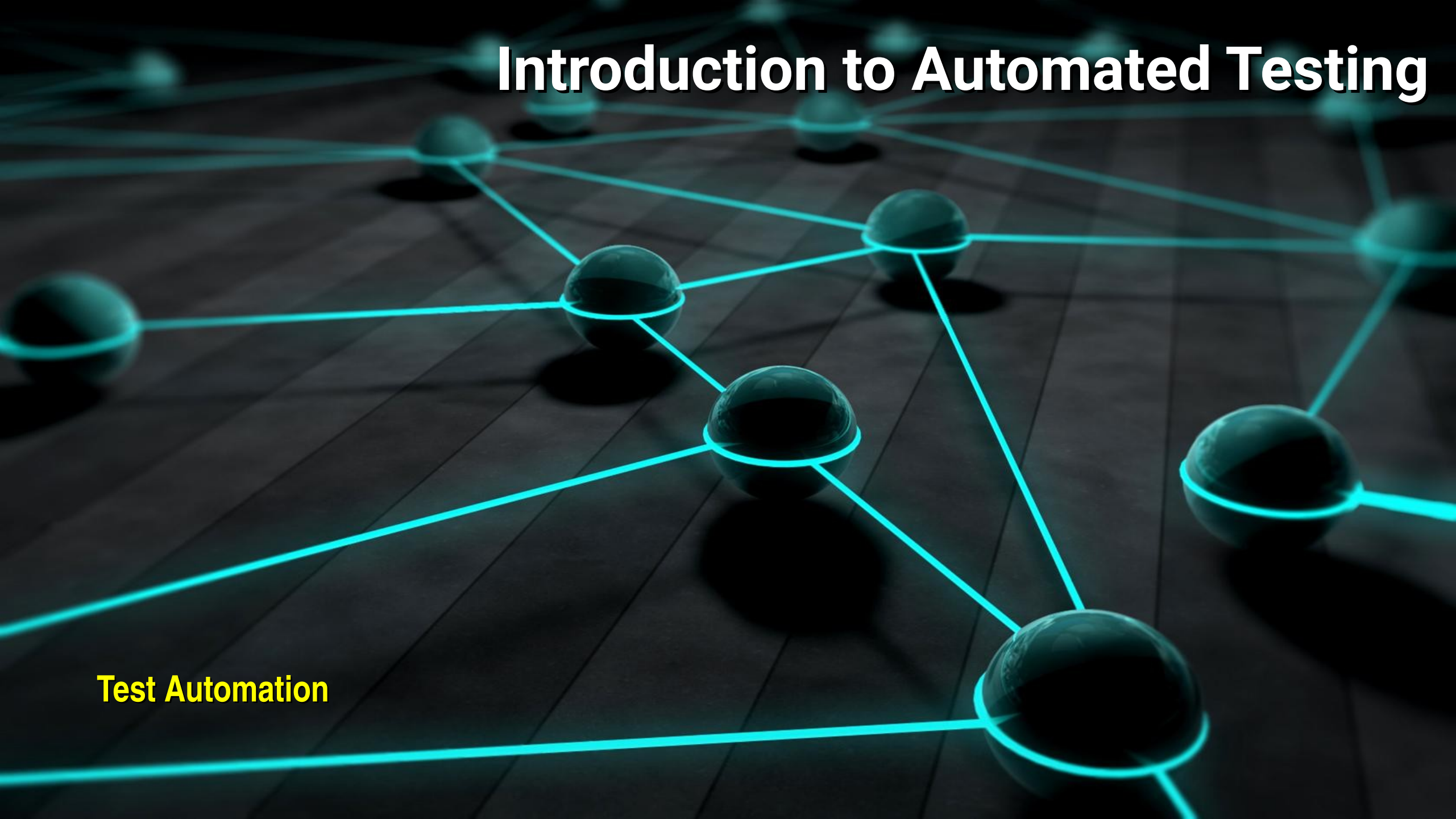
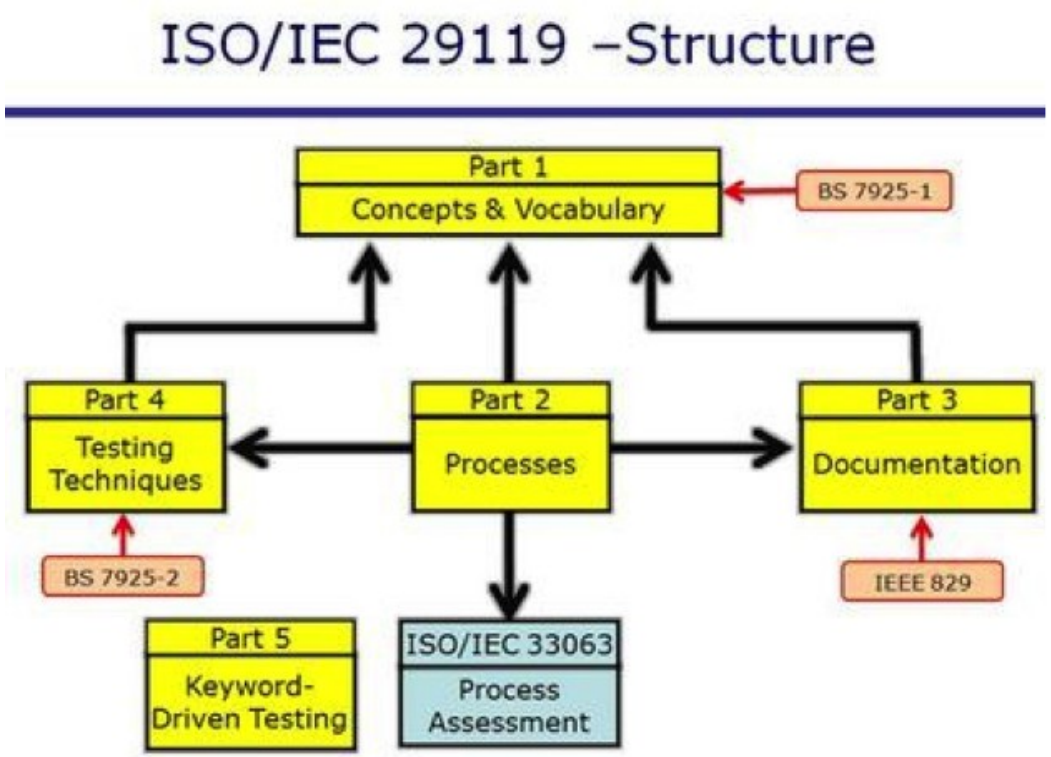
Introduction to Automated Testing



Test Automation

Standardization

- Test automation is starting to become standardized
- Published bodies of knowledge:
  - ISTQB CTAL-TAE v2.0: Test Automation Engineering (2024)
  - ISTQB CT-TAS v1.0: Test Automation Strategy (2024)
  - ISO/IEC/IEEE 29119: Software Testing (Parts 1–5)
  - Meszaros: xUnit Test Patterns (test smells)
  - Humble & Farley: Continuous Delivery
  - Graham & Fewster: Experiences of Test Automation
  - DORA: DevOps Research and Assessment metrics



Certification Example

ISTQB® Certified Tester Advanced Level Test Automation Engineering (CTAL-TAE)

Introduction and Objectives for Test Automation	Preparing for Test Automation	Test Automation Architecture	Implementing Test Automation	Implementing and Deployment Strategies for Test Automation	Test Automation Reporting and Metrics	Verifying the Test Automation Solution	Continuous Improvement
Purpose of Test Automation	Understand the Configuration to Enable Test Automation	Design Concepts Leveraged in Test Automation	Test Automation Development	Integration to CI/ CD Pipelines	Collection, Analysis and Reporting of Test Automation Data	Verification of the Test Automation Infrastructure	Continuous Improvement Opportunities for Test Automation
Test Automation in the Software Development Lifecycle	Evaluation Process for Selecting the Right Tools and Strategies		Risks Associated with Test Automation Development				
			Test Automation Solution Maintainability				

Planning Automation

- Common planning failure
  - "We're going to automate the regression suite."
- Why this fails
  - No measurable objective
  - No definition of done
  - No selection criteria
  - No maintenance plan
  - No way to tell whether it worked

Planning Automation

- Step 1: Define a measurable objective
  - Weak objectives
    - *"Automate the regression suite"*
    - *"Achieve 80% automation"*
    - *"Find more bugs"*
  - Strong objectives
    - *Reduce regression cycle from 5 days to 2 hours*
    - *Developer feedback on every commit in under 10 minutes*
    - *Free 30% of manual tester time for exploratory testing*
    - *Enable weekly releases instead of quarterly*

Automation is good at	Humans are good at
Repeating known checks	Discovering unknown problems
Speed and scale	Judgment and context
Consistency	Usability and aesthetics
Regression protection	Exploratory investigation
Combinatorial data	Asking "should it do this?"

Planning Automation

- Step 2: Assess testability first
  - Design for testability before tool selection.
  - No tool choice compensates for an untestable system.
  - Two questions define testability:
    - *Control: can I put the system into a known state?*
    - *Observation: can I see what happened?*
- IEEE definition of testability
  - Being able to write a predicate for each test condition that evaluates to pass/fail result
  - Or, in the case of continuous variables, can be expressed in terms of a range of allowed values
    - *Usually expressed as mean, standard deviation, confidence interval or confidence measure*

Planning Automation

- Control points
  - Can the test drive the system into a known state?
    - *Seed the database to a known fixture*
    - *Inject or freeze the clock*
    - *Toggle feature flags*
    - *Bypass login via a test API or token*
    - *Reset state between tests*
    - *Force error conditions and edge cases*
  - Without control points, tests are guesses about state

Planning Automation

- Observation points
  - Can the test see what happened?
    - *Stable, semantic element IDs (data-testid, not div > div:nth-child(3))*
    - *Structured logs*
    - *A query API to inspect state*
    - *Health and diagnostics endpoints*
    - *Access to the database or message queue for verification*
  - Without observation points, tests cannot accurately assess a pass/fail result

Planning Automation

- Step 3: Selection Criteriac
  - Do not automate everything
    - *Score candidates on criteria like those to the right, but use YOUR criteria*
    - *This is an engineering decision*
  - What not to automate
    - *One-off tests you will run once*
    - *Features still churning in design*
    - *Usability, look-and-feel, "does this feel right"*
    - *Exploratory testing*
    - *Anything where the oracle is human judgment*
    - *Tests that cost more to maintain than to run manually*

Factor	Favours automation
Execution frequency	Runs often
Feature stability	Unlikely to change
Determinism	Same input → same output
Business risk	High cost of failure
Manual cost	Slow or tedious by hand
Data availability	Test data is obtainable
Lifespan	Test stays relevant for years

Planning Automation

- Step 4: Choose the test layer before committing to a tool
  - Wrong order: "We picked Selenium. Now what do we test?"
  - Right order: "What needs verifying, at which layer, how fast?"
    - *Unit: fastest, most numerous, cheapest to maintain*
    - *Integration / API: moderate speed, verifies contracts*
    - *E2E / UI: slowest, fewest, most brittle*
  - The tool follows from the test layer decision

Planning Automation

- Step 5: Environment and test data strategy
  - In practice, this is the actual bottleneck on most real projects.
  - Decide before you start:
    - *Where do tests run? Who owns those environments?*
    - *How is test data created, reset, and isolated?*
    - *Shared database or per-run isolation?*
    - *How do you handle third-party dependencies?*
    - *Who fixes the environment when it breaks at 2am?*
  - Most automation programs stall on issues of responsibility, not tooling

Planning Automation

- Step 6: Pilot before deploying
  - Determine a representative set of tests to automate
  - Explicit success criteria defined in advance
    - *What exactly does a pass or fail look like? How do we measure it?*
  - Genuine willingness to abandon the tool
    - *The tool may not be the right choice in terms of performance characteristics*
    - *Costs associated with tool use may be higher than expected*
    - *The tool may not integrate into existing processes*
  - Measure the real cost, including setup and debugging
  - A pilot that you decide can’t fail is not a pilot.
    - *Fail early, fail cheap compared to costs of failures in production*

Planning Automation

- Step 7: Budget maintenance during planning
  - Automation maintenance typically runs 20 to 40% of the initial build cost, per year.
    - *A plan without this line item will collapse in year two.*
  - Maintenance drivers:
    - *Application UI and API changes*
    - *Test data drift*
    - *Framework and dependency upgrades*
    - *Flaky test investigation*
    - *Environment changes*

Planning Automation

- Planning checklist
  - Measurable objective, not "automate the regression suite"
  - Testability assessed: control and observation points exist
  - Test selection criteria defined and applied
  - Target layer test domain defined
  - Environment and test data strategy documented
  - Tool chosen after planning
  - Pilot completed with objective evaluation
  - Maintenance budgeted as a recurring cost
  - Skills and ownership assigned

Testing the Automation

- Tests are code and code has bugs
- Two failure modes
  - False positive: test fails, system is fine
    - *Costs credibility*
    - *Wastes investigation time*
    - *Enough of these and the team stops responding to failures*
  - False negative: test passes, bug ships
    - *Costs money*
    - *Far more dangerous because it is invisible in a release*
    - *You never find out you had one until production breaks.*
    - *A suite full of false negatives looks perfectly healthy*
    - *Remember this is called predictive validity.*

Common Mistakes

- Automation test errors come from three sources
  - Errors in the test cases used
    - *This is a problem that is addressed by software testers in their test specification process*
    - *This is not generally within the scope of the automation team*
  - Errors in the test execution
    - *Usually testing the wrong code or using the wrong data*
    - *This is addressed by rigorous test planning and following well defined testing procedures*
  - Errors in the code of the testing tool or testing scripts
    - *This is addressed by “testing the test code”*

Testing the Automation

- Testing the test code
  - A mock of the actual system is provided
  - It returns exactly the expected value for every test case
    - *This means that all of the test cases have passed vacuously*
  - It also includes a failure response for each test case
    - *Each test is run twice, one is a pass, the other is a fail*
  - The test automation tool is run against the test mock
  - We look for
    - *Tests that fail that should have passed.*
    - *Tests that passed that should have failed*
  - These indicate places in the test scripts that either
    - *Did not run the test correctly or*
    - *Did not process the result correctly*

Testing the Automation

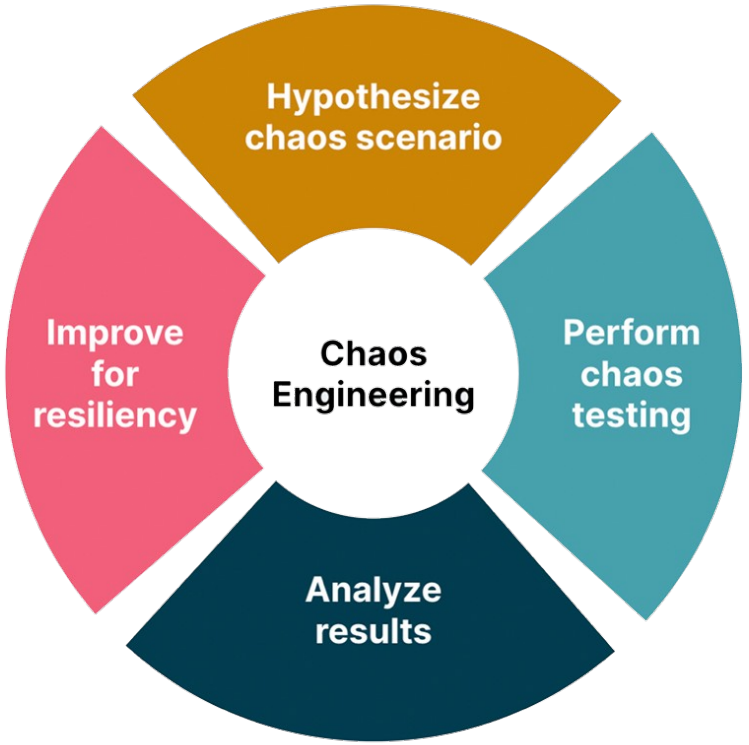
- Mutation testing
  - The direct answer to "do my tests work?"
  - Tool injects small faults ("mutants") into production code
    - *> becomes >=*
    - *+ becomes -*
    - *return x becomes return null*
  - Runs your test suite against each mutant
    - *Mutant killed = a test caught it*
    - *Mutant survived = a gap in your assertions*
  - Mutation score = killed / total mutants

Mutation Testing Tools

Language	Tool
Java	**Pitest** (Maven / Gradle plugin)
Python	**mutmut, cosmic-ray**
Go	**go-mutesting**
JavaScript / TS	**Stryker**

Testing the Automation

- Verify the test can fail
  - A test that has never failed has never been validated.
- Three practices:
  - Write the failing test first
    - *Proves the test is actually testing the thing it is supposed to be testing*
  - Break the system deliberately
    - *Introduce a known defect.*
    - *Confirm the suite catches it*
  - Chaos engineering
    - *Injecting a fault into any part of the stack that an application requires to run (compute, storage, networking, and application infrastructure)*
    - *Used a lot in resilience testing*



Testing the Automation

- Seeded defect studies
  - Deliberately inject a set of known defects, then measure:
  - Defect Detection Percentage gives you an empirical measure of suite effectiveness
  - Useful as a periodic health check, not a daily practice

$$\text{DDP} = \frac{\text{defects found by suite}}{\text{defects seeded}} \times 100\%$$

Testing the Automation

- Coverage
  - Measures execution, not verification
- The test on the right
  - Will execute every line of the function under test
  - But there is no verification step
    - *We don’t know if calculateTotal() executed correctly*
    - *Reports 100% coverage. But it can never fail*
  - Coverage tells you what code was executed.
    - *It tells you nothing about what was verified.*
    - *40% coverage means you definitely have untested code*
    - *100% coverage but you might have zero effective tests*
  - Use coverage to find gaps
    - *Never use it as a quality target.*

```java
@Test
void testCalculateTotal() {
    Order order = new Order();
    order.addItem(new Item("Widget", 9.99), 3);
    order.calculateTotal();
    // no assertion
}
```

Testing the Automation

- Test code Is production code
- Apply the same engineering standards:
  - Version control (same repo as the code under test)
  - Code review: reviewed as carefully as production code
  - Coding standards and linting
  - Static analysis
    - *Use scanners like SonarQube on test code*
  - Refactoring
  - Its own unit tests for framework and helper code
    - *Meta-testing, use with the test mock discussed earlier*

Testing the Automation

- Be on the lookout for test smells

Smell	Symptom
Assertion Roulette	Many unlabelled assertions — which one failed?
Mystery Guest	Test depends on external file or data you can't see
Fragile Test	Breaks on unrelated changes
Erratic Test	Passes and fails without code changes (flaky)
Eager Test	One test verifies many behaviours
Conditional Test Logic	`if` / loops inside tests
Test Code Duplication	Copy-pasted setup everywhere

Testing the Automation

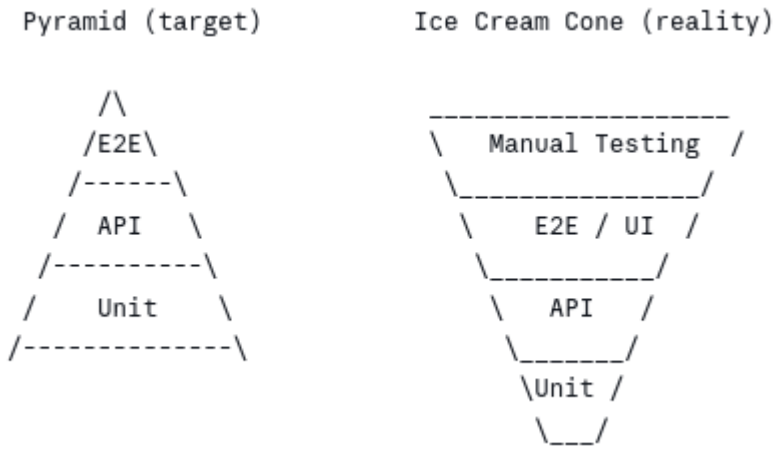
- Record keeping
  - Detect
    - *Track historical pass rate per test*
    - *Re-run analysis: same commit, different result*
  - Root causes
    - *Timing and race conditions*
    - *Shared mutable state between tests*
    - *Test ordering dependencies*
    - *Uncontrolled randomness or system clock*
    - *Real network calls*
  - Policy
    - *Quarantine with a deadline*
    - *Never "re-run until green"*
    - *Classify every failure*
    - *If more than ~25% of failures are test defects, the automation itself is the problem.*

Category	Meaning
Real defect	Automation did its job
Test defect	The test is wrong
Environment	Infrastructure failed
Data	Test data was wrong or missing

Common Mistakes

- Strategy-level mistakes

Mistake	Consequence
Automating everything	Huge maintenance burden, low value
Record-and-playback as strategy	Unmaintainable, breaks constantly
Inverted pyramid	Slow, flaky, expensive feedback
UI-testing what an API could verify	100× slower, 10× more brittle
Automating before testability exists	Permanent cost multiplier
Automation as a separate silo team	Disconnected from the code it tests



Common Mistakes

- Code-level mistakes
  - No abstraction layer
    - *No Page Object, no Screenplay, mass duplication of test code*
  - Brittle locators
    - *Tied to position or styling instead of stable IDs in pages*
  - Thread.sleep() instead of explicit waits
  - Test interdependence
    - *Test B only passes if Test A ran first*
  - Shared mutable state between tests
  - Assertion Roulette
    - *Twelve assertions, no messages*
  - No diagnostics on failure
    - *No screenshot, no logs, no failure report*

```java
// WRONG — flaky and slow
driver.findElement(By.id("submit")).click();
Thread.sleep(3000);
assertEquals("Success", driver.findElement(By.id("msg")).getText());

// RIGHT — waits exactly as long as needed
driver.findElement(By.id("submit")).click();
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.textToBe(By.id("msg"), "Success"));
```

```gherkin
# WRONG — imperative, UI-coupled, unreadable by business
Given I navigate to "/login"
When I type "user@test.com" into field "#username"
And I type "hunter2" into field "#password"
And I click the button with class ".btn-submit"

# RIGHT — declarative, business-readable, UI-independent
Given I am a registered customer
When I sign in with valid credentials
Then I should see my account dashboard
```

Common Mistakes

- Organizational mistakes
  - Tolerating flaky tests; @Ignore instead of fix
  - No maintenance budget resulting in accumulating automation debt
  - Tests that only run when someone remembers
  - Measuring the wrong things

Vanity metric	What it incentivizes
Number of tests automated	Writing many trivial tests
% of test cases automated	Automating easy, low-value cases
Code coverage %	Tests without assertions

Real metric	What it tells you
Escaped defect rate	Did automation catch what mattered?
Feedback time to developer	Is the loop fast enough to use?
Flake rate	Do people trust the build?
Maintenance effort %	Is this sustainable?

Automation Within QA Policy

- Common failure; automation exists only at the plan level
  - Re-invented per project, dies when its champion leaves
- Ownership models
  - Siloed model (common, problematic)
    - *Separate "automation team"*
    - *Receives builds over the wall*
    - *Always behind the development team*
    - *No influence over testability*
  - Whole-team model (Crispin & Gregory)
    - *Developers own unit and integration tests*
    - *Specialists build the framework and enable others*
    - *Quality is everyone's responsibility*
    - *Often called Agile Testing*

Per ISO/IEC/IEEE 29119 and ISTQB:

```
TEST POLICY        Organizational. Why we test.
     |             What quality means here.
     v
TEST STRATEGY      How we test across projects.
     |             *** AUTOMATION STRATEGY LIVES HERE ***
     v
TEST PLAN          Project-specific application.
```

Automation Within QA Policy

- Definition of Done
  - Automated tests at the appropriate level are part of Done
    - *They are not a follow-up ticket or add-on*
  - A story is not done when:
    - *"We'll automate the testing in the next sprint"*
    - *"The tests are in the automation backlog"*
    - *"QA will pick that up later"*
  - Deferred test automation is never written
    - *It often lacks a sense of urgency when it is not part of the definition of Done*

Automation Within QA Policy

- Governance questions
  - Who may quarantine a test, and what is the fix deadline?
  - Who may override a failed quality gate, and is it logged?
  - What happens when the build fails: stop the pipeine or keep going?
  - Who owns test environments and their uptime?
  - What is the SLA for fixing broken automation?
- Metrics that belong in a policy
  - Escaped defect rate (defects found in production)
  - Defect Detection Percentage
  - Feedback time to developer
  - Flake rate / test stability
  - Mean time to pass after a failed build
  - Maintenance effort as % of automation effort
  - Notice what is absent: test counts and coverage targets.

Automation Within QA Policy

- Regulated environments
- Automation carries extra obligations in regulated domains:
  - Traceability matrices
    - *Able to track requirement to corresponding tests and their results*
  - Evidence retention
    - *Auditable records of every run are maintained and searchable*
  - Tool qualification
    - *Proving the tools are fit for purpose*
  - Change control
    - *Test changes are controlled changes*
    - *Test changes can be rolled back*

A Simple Maturity Ladder

Level	Characteristics
1 — Ad hoc	Individual scripts, no standards, run manually
2 — Repeatable	Shared framework, version controlled, documented
3 — Integrated	Runs in CI on every commit, gates merges
4 — Optimized	Test selection, parallelization, flake management
5 — Strategic	Metrics drive decisions, automation informs design

DevOps, CI/CD and Operations

- The time budget forces the pyramid
  - Cannot fit a large E2E suite into a 10-minute commit stage.
    - *2,000 unit tests × 5ms = 10 seconds*
    - *2,000 E2E tests × 30s = 16 hours*
  - The pyramid is a consequence of caring about feedback speed.
- This is critical to a shift-left approach

The Organizing Principle: Fast Feedback

The deployment pipeline (Humble & Farley):

COMMIT STAGE	ACCEPTANCE STAGE	DEPLOY
< 10 minutes	30-60 minutes	on demand
Compile	API / integration	Smoke tests
Unit tests	Contract tests	Canary
Static analysis	Key E2E journeys	Monitoring
Quality gate	Performance	

DevOps, CI/CD and Operations

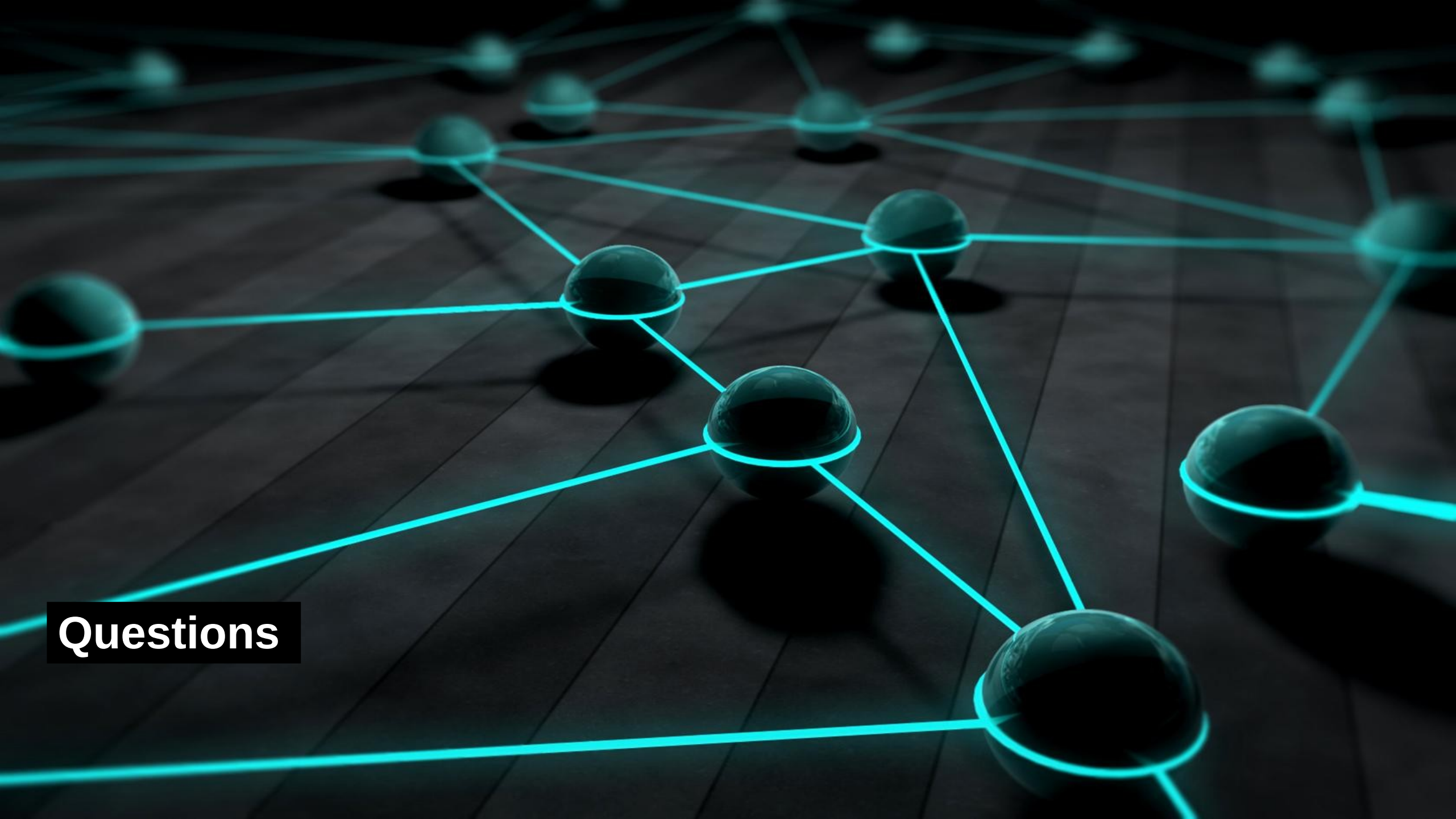
- Staying inside the budget
  - Parallelization: run tests concurrently across workers
  - Sharding: split the suite across machines
  - Test impact analysis: run only tests affected by the change
  - Tiered suites: smoke (5 min) / full (1 hr) / nightly (6 hr)
  - Fail fast: order tests so likely failures run first
- Environments as code
  - Ephemeral, reproducible, disposable.
  - Infrastructure as Code (Terraform, Pulumi)
  - Containers (Docker, Kubernetes)
  - Fresh environment per pipeline run
  - Destroyed after use

DevOps, CI/CD and Operations

- Test data in pipelines
- Decisions to make and record:
  - Seeding: known fixtures loaded before each run
  - Isolation: shared database vs per-run schema vs per-test transaction
    - *Isolation costs setup time, sharing costs determinism.*
  - Reset: how state returns to baseline
  - Synthetic generation: creating realistic data at volume
  - Masking: anonymizing production data for privacy compliance

DevOps, CI/CD and Operations

- Shift right
  - Some verification is cheaper and more truthful in production:
    - *Synthetic monitoring: the smoke suite, running continuously*
  - Canary deployments: release to 1% and watch
  - Feature flags: decouple deploy from release
  - Observability: logs, metrics, traces as test oracles
  - Chaos engineering: inject failure deliberately
- Testing does not stop at deployment.
  - This overlaps with monitoring and detective controls in operations
  - But in DevOps, the feedback from the Ops environment feeds back into development


Questions